

# Rockchip Linux Low Power Wi-Fi Developer Guide

---

文件标识：RK-KF-YF-A59

发布版本：V1.0.0

日期：2025-08-08

文件密级：☐绝密 ☐秘密 ☐内部资料 ☒公开

## 免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

## 商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自所有者所有。

## 版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址：福建省福州市铜盘路软件园A区18号

网址：[www.rock-chips.com](http://www.rock-chips.com)

客户服务电话：+86-4007-700-590

客户服务传真：+86-591-83951833

客户服务邮箱：[fae@rock-chips.com](mailto:fae@rock-chips.com)

前言

概述

Rockchip Linux平台低功耗Wi-Fi开发移植调试

产品版本

| 芯片名称                                  | 内核版本           |
|---------------------------------------|----------------|
| RV1106/RV1103/RV1106B/RV1103B/RV1126B | Linux 5.10/6.1 |

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

硬件开发工程师

修订记录

| 版本号    | 作者 | 修改日期       | 修改说明 |
|--------|----|------------|------|
| V1.0.0 | xy | 2025-08-08 | 初始版本 |

## Rockchip Linux Low Power Wi-Fi Developer Guide

1. 简介
2. 框架介绍
3. 流程介绍
4. 方案介绍
  - 4.1 方案总体介绍
    - 4.1.1 RK主控
    - 4.1.2 Wi-Fi MCU
  - 4.2 RK主控芯片侧配置
  - 4.3 RK962方案
  - 4.4 海思Hi3861L/Hi3853方案
  - 4.5 爱科微AIC8800M方案
  - 4.6 高拓ATBM6441/6461方案
5. 调试指导

# 1. 简介

## 低功耗WiFi技术

- **超低功耗：**通过实现超低的运行功耗，显著延长设备的电池续航时间，满足各种需要长时间待机的应用场景。
- **独立网络处理：**低功耗Wi-Fi模块能够脱离主芯片独立运作，内部有独立ROM/RAM/网络协议栈，能够脱离主控芯片独立运行，并直接与云端服务器通信，确保主设备在深度休眠时依然保持网络连接。
- **远程唤醒：**支持标准化的网络唤醒指令，可远程唤醒处于休眠状态的主控芯片，无需人工现场干预。
- **快速网络恢复：**主控芯片被唤醒后，Wi-Fi模块可实现百毫秒级网络连接恢复，确保设备能迅速响应网络指令和用户操作。
- **定制化MCU及接口：**Wi-Fi模块内置的MCU提供丰富的硬件接口（如GPIO、I2C等），支持外接PIR传感器、按键、充电管理等外设的定制化开发，以满足特定应用需求。

## 目前，市场上主要存在两种低功耗Wi-Fi技术方案

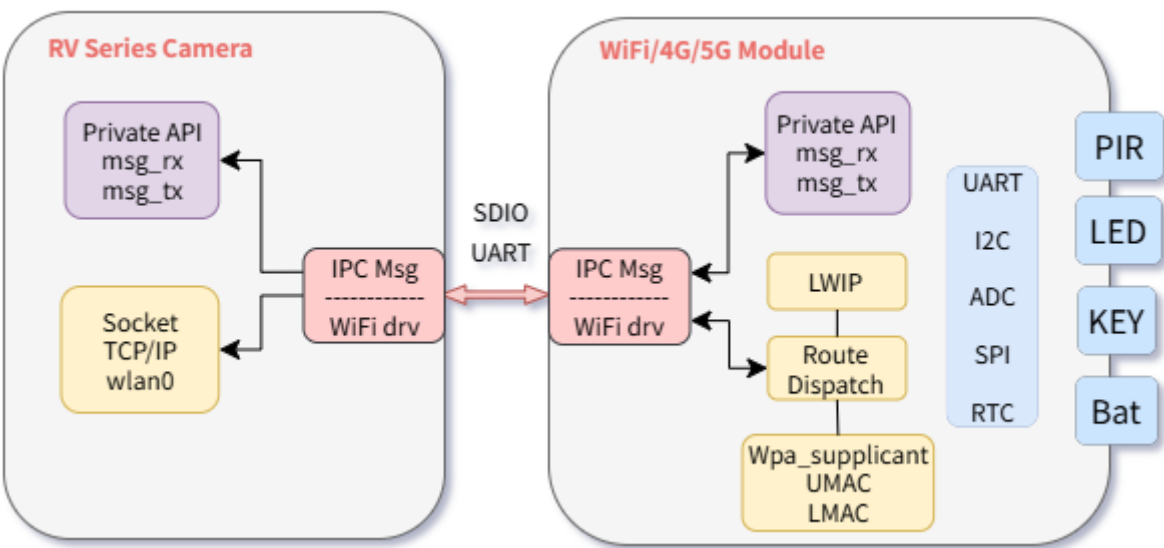
- **集成MCU的Wi-Fi模块**

此类模块内部集成了MCU、RAM和ROM，可独立完整地运行无线网络协议栈（如WPA）和TCP/IP协议栈（如LwIP）。它支持二次开发，允许用户编程实现独立的外设控制与网络任务，灵活性高，是当前主流方案。

- **无MCU的Wi-Fi模块**

此类模块不含独立的MCU，仅能执行简单的网络保活程序，通常需要搭配一颗外部MCU来协同工作，导致方案整体成本偏高，因此较少被采用。

# 2. 框架介绍

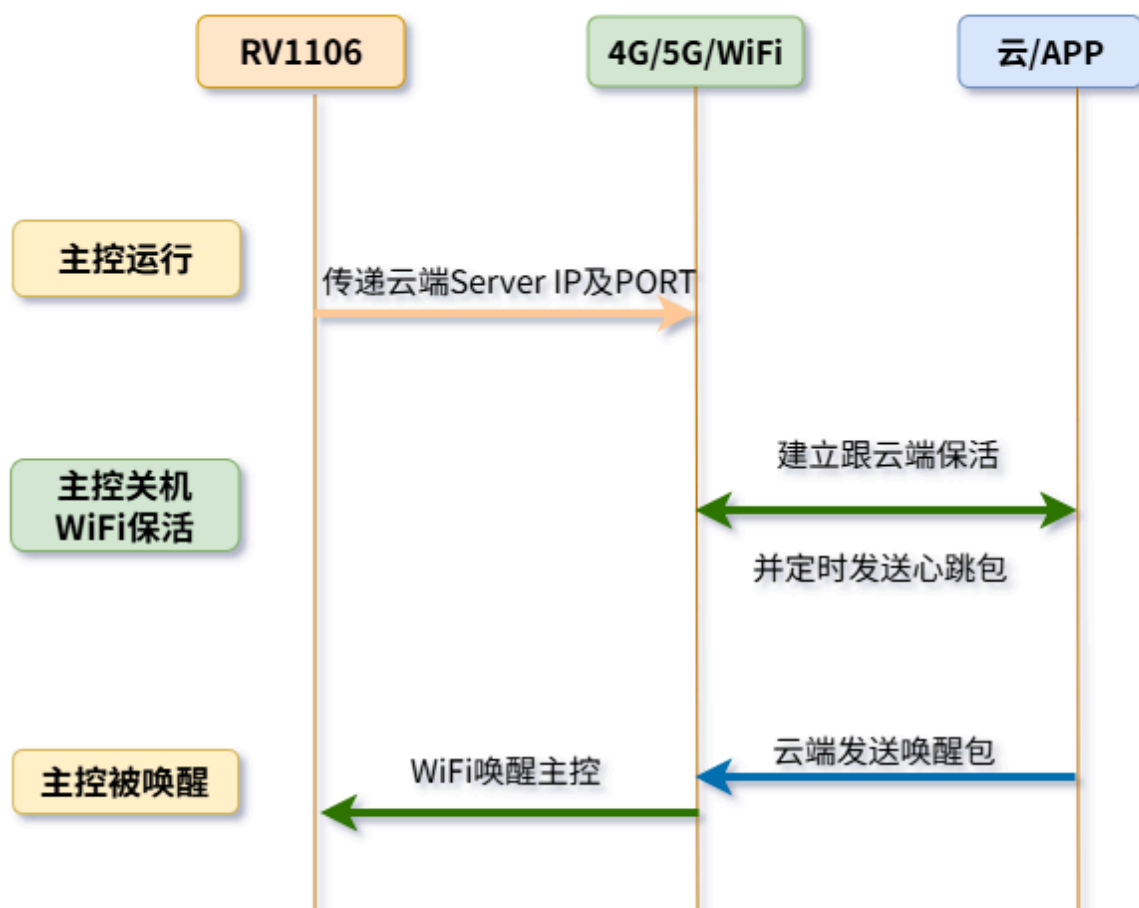


低功耗方案都会实现类似上图的架构：

- **私有通道 (Private API):** 主控与Wi-Fi模块之间通过私有的IPC消息机制进行通信，用于传输双方预定义的指令，如网络配置、保活参数设置、进入/退出低功耗模式、获取PIR状态或电池电量等。
- **标准网络接口:** Wi-Fi模块向主控提供一个标准的类以太网驱动，生成一个网络接口（如 wlan0）。虽然名为无线网卡，但其完整的无线协议栈运行在模块内部。主控应用程序可通过标准的Socket API 进行网络数据收发。
- **路由/分发 (Route/Dispatch):** Wi-Fi模块内部的数据分发组件。它根据网络数据的目标端口号等规则，决定数据包是交由主控的协议栈处理，还是在模块内部的LwIP协议栈处理。
- **内部协议栈:** Wi-Fi模块内置了完整的WPA Supplicant、UMAC、LMAC等组件，扫描、连接、认证等无线网络操作均在模块内部完成，不消耗主控的系统资源。
- **独立保活与唤醒:** 在主控休眠或关机期间，Wi-Fi模块通过内置的LwIP协议栈独立与远端服务器维持心跳连接（保活），并监听网络唤醒指令，从而实现超低功耗待机。
- **外设独立管理:** Wi-Fi模块可独立管理PIR传感器、LED、按键、充电IC、电量计等外设，在主控休眠时处理相关逻辑，无需主控参与。

### 3. 流程介绍

整体流程介绍：下图展示一个摄像头从工作到低功耗休眠再到工作的完整流程。



#### 正常工作状态

- IPC设备启动后，与云端服务建立连接并进行实时视频推流。
- 当推流结束或云端主动断开连接后，IPC应用将云端保活服务所需的IP地址和端口号等参数发送给Wi-Fi模块。

### 休眠触发

- IPC设备可通过物理按键、摄像头程序逻辑（例如空闲时段检测）等方式被触发进入休眠状态，或者由Wi-Fi模块远程控制IPC设备进入下电或休眠状态。

### 进入休眠前的准备

- 在IPC主控进入休眠前，Wi-Fi模块使用预先配置的保活参数，独立与云端服务器建立并维持一个TCP保活连接。
- Wi-Fi模块周期性地向云端发送心跳包，以维持设备在线状态，告知云端设备正处于可唤醒的低功耗模式。

### 设备休眠状态

- IPC主控及大部分外设下电，此时仅Wi-Fi模块维持极低功耗运行，并持续与云端进行心跳通信。

### 设备唤醒过程

- 当云端需要访问IPC时，通过保活连接向Wi-Fi模块发送特定的唤醒指令。
- Wi-Fi模块接收到指令后，通过GPIO信号唤醒IPC主控（例如，控制电源芯片为IPC主控重新上电）。

### 退出休眠并恢复正常工作

- IPC主控启动后，由于Wi-Fi模块一直在线且网络已连接，设备可以快速与云端重新建立业务连接，恢复视频推流等正常工作。

## 4. 方案介绍

---

目前支持的方案有：

- 瑞芯微：RK962
- 海思：Hi3861L、Hi3853
- 高拓：ATBM6441、ATBM6461
- 爱科微：AIC8800M

### 4.1 方案总体介绍

开发工作主要分为两大部分：RK主控侧 和 Wi-Fi MCU侧。

#### 4.1.1 RK主控

- Wi-Fi驱动

需特别注意，由于低功耗Wi-Fi模块内部已集成完整的无线协议栈，其提供给主控的是一个类以太网（Ethernet-like）或USB网卡驱动。因此，RK主控侧无需加载标准的cfg80211/mac80211无线框架，其驱动模型与传统Wi-Fi有本质区别。

- Wi-Fi配网

同理，由于驱动模型的差异，配网操作不再通过标准的wpa\_supplicant工具。取而代之的是各Wi-Fi方案商提供的私有应用程序或指令集，例如海思的hisi\_tools、高拓的atbm\_iodot\_supplicant或Rockchip方案的rkwifi\_server等。

- **保活及私有命令**

配置保活参数、进出低功耗模式、控制PIR、获取电池电量等所有与模块交互的私有功能，均通过上述方案商提供的特定应用进行控制。

## 4.1.2 Wi-Fi MCU

- **Wi-Fi驱动与协议栈**

Wi-Fi模块的固件完整实现了从LMAC、UMAC到wpa\_supplicant的无线协议栈，并集成了LwIP网络协议栈，使其能作为一个完全独立的网络设备运行。其内部的数据分发机制会根据端口号等规则决定数据包的流向（上报给主控或内部处理）。

- **MCU的外设开发**

在低功耗场景下，当RK主控断电休眠时，充电管理、电量检测、PIR及按键检测等功能必须由Wi-Fi MCU接管。因此，相关的驱动和应用程序也需要移植或开发在MCU上。

- **跟RK主控交互的模块**

此模块负责解析并执行由主控侧应用（如hisi\_tools）下发过来的私有命令。

**注意：**RK主要工作是适配运行在主控上的类以太网驱动。Wi-Fi MCU固件及其相关开发（包括无线协议栈、外设驱动、与主控的交互逻辑等），均由Wi-Fi方案商（芯片原厂或模组厂）提供技术支持。

## 4.2 RK主控芯片侧配置

首先参考文档：Rockchip\_Developer\_Guide\_Linux\_Wi-Fi\_BT\_CN.pdf

### BoardConfig 配置

以RV1106 EVB2低功耗开发板为例，使用如下配置：

```
project/cfg/BoardConfig_BatteryIPC/BoardConfig-SPI_NOR-NONE-RV1106_EVB2_V11-BAT_IPC.mk
```

```
#添加Wi-Fi配置: rv1106-sdiowifi.config
export RK_KERNEL_DEFCONFIG_FRAGMENT="rv1106-tb.config rv1106-battery-ipc.config
rv1106-sdiowifi.config"

#RK方案: RK962
#海思方案: HI3861L
#高拓方案: ATBM6441
export RK_ENABLE_WIFI=y
export RK_ENABLE_WIFI_CHIP="HI3861L"
```

### 开发板DTS配置

```
sysdrv/source/kernel/arch/arm/boot/dts/rv11xx.dts
&sdio {
    max-frequency = <30000000>;
    no-sd;
```

```

no-mmc;
//Wi-Fi如果使用4线，则需要改为4
bus-width = <1>;
cap-sd-highspeed;
cap-sdio-irq;
keep-power-in-suspend;
non-removable;
pinctrl-names = "default";
//根据硬件二选一，以RV1106芯片为例，其他芯片类似
//SDIO有两组GPIO可以选择，根据硬件的实际接发(对照原理图)
//选择其中一个即可，sdmmc1mX_XXX对应的gpio号参考如下文件：
//sysdrv/source/kernel/arch/arm/boot/dts/
//rv1106-pinctrl.dtsi
pinctrl-0 = <&sdmmc1m0_cmd &sdmmc1m0_clk &sdmmc1m0_bus4>;
//pinctrl-0 = <&sdmmc1m1_cmd &sdmmc1m1_clk &sdmmc1m1_bus4>;
status = "okay";
};

```

### Wi-Fi 类以太网驱动

- RK主控驱动：sysdrv/drv\_ko/wifi/rk96x
- 海思主控驱动：sysdrv/drv\_ko/wifi/hichannel
- 高拓主控驱动：sysdrv/drv\_ko/wifi/atbm6441 or atbm6461
- 爱科微主控驱动：sysdrv/drv\_ko/wifi/aic8800\_netdrv

## 4.3 RK962方案

RK962是一款集成Wi-Fi与蓝牙功能的高性价比低功耗SoC。它支持在特定连接模式下（如DTIM10 @3.3V）实现250uA的超低功耗，并内置MCU以支持外设开发及BLE辅助配网。由于篇幅限制，详细适配资料请联系业务或FAE获取。

## 4.4 海思Hi3861L/Hi3853方案

首先，请从Rockchip FTP（[请联系FAE获取](#)）获取海思模组相关资料，路径为：/11-Linux平台/WIFI低功耗方案/海思/2023。

### 1. 强烈建议首先详细阅读SDK内的指导文档

Hi3861L ReleaseDoc/zh/02 软件资料/05 特性开发指导/Hi3861LV100 Hichannel 使用指南.pdf

Hi3861L ReleaseDoc/zh/02 软件资料/05 特性开发指导/Hi3861LV100 Hichannel 适配常见问题 FAQ.pdf

Hi3861L ReleaseDoc/zh/02 软件资料/05 特性开发指导/Hi3861V100/Hi3861LV100 低功耗开发指南.pdf

Hi3861L ReleaseDoc/zh/02 软件资料/02 平台开发指导

### 2. 私有接口说明

海思原厂SDK中提供了一套基于sample\_link（服务端）和sample\_cli（客户端）的私有通信Demo。

服务端: `HiChannel_V1.0.0.1/host/app/demo_linux/app/client`

客户端: `HiChannel_V1.0.0.1/host/app/demo_linux/app/link`

在RK平台上, 我们通常使用如VLink (深圳市纬联) 等模组厂商基于海思SDK二次开发的版本。其结构分为:

**Wi-Fi侧:**

**SDK包:** `Hi3861LV100R001C00SPC032.tar.rk20240110.bz2` (Vlink二次开发版)

**核心代码:** `app/tdemo/`目录是与主控`rkwifi_server`交互的核心逻辑。

目录结构:

```
app/tdemo/  
src  
├─ app_main.c  
├─ vlink_deepsleep_proc.c          //休眠相关  
├─ vlink_hichannel_channel.c      //Hichannel通信  
├─ vlink_hichannel_main.c         //Hichannel入口  
├─ vlink_hichannel_util.c  
└─ vlink_local_server_keepalive.c //保活相关
```

关键代码分析:

```
//Wi-Fi模块端SDK接收消息的函数  
Hi3861LV100R001C00SPC032/app/tdemo/src目录  
//接收主控端的各类命令并进行处理  
hi_channel_rx_callback()  
{  
    //解析hi_channel_send_to_dev发过来消息, 并处理  
}  
  
//Wi-Fi模块端给主控发消息:  
Hi3861LV100R001C00SPC032/include/base/hi_channel_dev.h  
unsigned int hi_channel_send_to_host(char* buf, int length)  
{  
    //比如发送连接成功的事件/IP地址等数据给主控  
}
```

**主控侧:**

**代码路径:** `SDK/project/app/wifi_app/wifi/`, 该部分是基于海思及模组厂提供的`demo_linux`应用代码进行的二次开发。

**抽象层:** `rk_wifi_hal.c` //实现通用的接口比如打开/连接Wi-Fi等操作

**具体实现层:** `rk_hisi_wifi.c` //封装海思的`demo_linux`

**测试入口:** `test/rk_wifi_test.c`

```
//发送:  
//project/app/wifi_app/wifi/src/hisi_tools/vlinktest/vlink_hichannel_main.c  
//私有命令通过这里发给Wi-Fi模块  
hi_s32 vlink_sample_str_cmd_process(hi_void *pmsg)  
{  
    //命令:  
    //配网: netcfg
```

```

//获取IP: getip
//... ...
}

//接收:
//首先调用hisi_main()函数后会注册一个callback函数
hi_channel_register_rx_cb(sample_link_rec_cb);

//这个函数接收来自Wi-Fi模块发来的信息
void sample_link_rec_cb(unsigned char *msg_data, int len)
{
    //这里解析hi_channel_send_to_host发来的信息
}

```

可以根据此通信规则，自行添加更多私有命令。

### 3. 根据文档编译Wi-Fi模块的固件

参考文档/11-Linux平台/WIFI低功耗方案/海思/2023/Hi3861L ReleaseDoc/zh/02 软件资料/02 平台开发指导。

### 4. 烧录

自制的板子烧录参考：

/11-Linux平台/WIFI低功耗方案/海思/2023/Hi3861L ReleaseDoc/zh/04 工具资料

EVB板烧录：

烧写工具：/11-Linux平台/WIFI低功耗方案/海思/2023/Hi3861L Tools

烧录参考：/11-Linux平台/WIFI低功耗方案/海思/2023/WiFi MCU固件升级

### 5. 加载ko

通过上面的配置编译烧录，主机在/oem/usr/ko生成hichannel.ko，开机加载主控驱动的命令为：

```

# 注意：hi_rk_irq_gpio为海思芯片与主控连接的中断脚的index
# GPIO index计算方法：
# GPIOx_yz hi_rk_irq_gpio ( 32 * x + 8 * (y - A) + z )
# 假设GPIO1_B3: 32 * 1 + 8 * (B - A) + 3 = 43
# 加载ko
# insmod /oem/usr/ko/hichannel.ko hi_rk_irq_gpio=40

```

**注意：**Hi3853的SDK结构跟3861完全不同，但原理都是一样的，本文档提供只提供一个主机侧的驱动移植补丁：

```

---
a/WS53_1.10_T5_release/application/samples/wifi/syschannel_host/linux/base.mak
+++
b/WS53_1.10_T5_release/application/samples/wifi/syschannel_host/linux/base.mak
@@ -6,7 +6,7 @@ SDK_DIR := $(CURDIR)
#+++++
#      config
#+++++
-CFG_HI_TOOLCHAINS_NAME = arm-himix100-linux
+CFG_HI_TOOLCHAINS_NAME = arm-rockchip830-linux-uclibcgnueabihf
#CFG_HI_TOOLCHAINS_NAME = x86_64-linux-gnu
#+++++

```

```

#      TOOL CHAINS

--- a/WS53_1.10_T5_release/middleware/utils/hcc/host/hcc_sdio_host.c
+++ b/WS53_1.10_T5_release/middleware/utils/hcc/host/hcc_sdio_host.c
@@ -465,7 +465,7 @@ td_s32 sdio_rescan_device(hcc_bus *pst_bus)
    #if defined(CONFIG_JZMMC_V12_MMC1) || defined(CONFIG_MMC_SDHCI_INGENIC)
        jzmmc_manual_detect(1, 1);
    #else
-    hisi_sdio_rescan(1);
+    //hisi_sdio_rescan(1);
    #endif

    for (i = 0; i < MAX_SDIO_RESCAN_NUM; i++) {
@@ -2082,7 +2082,7 @@ static td_s32 sdio_detectcard_change(td_void)
    #if defined(CONFIG_JZMMC_V12_MMC1) || defined(CONFIG_MMC_SDHCI_INGENIC)
        jzmmc_manual_detect(1, 1);
    #else
-    hisi_sdio_rescan(1);
+    //hisi_sdio_rescan(1);
    #endif
    #endif
    return EXT_ERR_SUCCESS;

--- a/WS53_1.10_T5_release/middleware/utils/syschannel/syschannel_host/Makefile
+++ b/WS53_1.10_T5_release/middleware/utils/syschannel/syschannel_host/Makefile
@@ -33,6 +33,14 @@ WLAN_CFLAGS += -DETH_PAD_SIZE=2
CROSS_COMPILE ?= mips-linux-gnu-
endif

+ARCH = arm
+KDIR = /path/sysdrv/source/objs_kernel
+WLAN_CFLAGS += -DSYSCHANNEL_LITTLE_ENDIAN=1
+WLAN_CFLAGS += -DSYSCHANNEL_BIG_ENDIAN=2
+WLAN_CFLAGS += -DSYSCHANNEL_ENDIAN=SYSCHANNEL_LITTLE_ENDIAN
+WLAN_CFLAGS += -DETH_PAD_SIZE=2
+CROSS_COMPILE = arm-rockchip830-linux-uclicginueabihf-
+
MAIN_PATH :=
OAL_PATH := oal/linux/
NETLINK_PATH := wal/linux/
@@ -46,7 +54,7 @@ OSAL_MEDIA_PATH :=
../../../../kernel/osal/src/linux/kernel/media/
HCC_PATH := ../../hcc/comm/
HCC_DRV_PATH := ../../hcc/host/
TOP_DIR := $(WLAN_DIR)/../../../../..
-WLAN_CFLAGS += -std=gnu99 -Wno-declaration-after-statement -fsigned-char -freg-
struct-return -Wtrampolines -Wfloat-equal -fvisibility=hidden
+WLAN_CFLAGS += -std=gnu99 -Wno-declaration-after-statement -fsigned-char -freg-
struct-return -Wtrampolines -Wfloat-equal
ifeq ($(CFG_3518EV300), y)
WLAN_CFLAGS += -Wdate-time
endif

---
a/WS53_1.10_T5_release/middleware/utils/syschannel/syschannel_host/oal/linux/oal_
netdev.h

```

```

+++
b/WS53_1.10_T5_release/middleware/utils/syschannel/syschannel_host/oal/linux/oal_
netdev.h
@@ -17,7 +17,7 @@ extern "C" {
    #define OAL_IFF_RUNNING                IFF_RUNNING
    #define OAL_IF_NAME_SIZE                16

-#define oal_netdevice_destructor(_pst_dev)      ((_pst_dev)->destructor)
+#define oal_netdevice_destructor(_pst_dev)      ((_pst_dev)-
>priv_destructor)
    #define oal_netdevice_qdisc(_pst_dev, pst_val)      ((_pst_dev)->qdisc =
(pst_val))
    #define oal_netdevice_ifalias(_pst_dev)      ((_pst_dev)->ifalias)
    #define oal_netdevice_watchdog_timeo(_pst_dev)      ((_pst_dev)->watchdog_timeo)

---
a/WS53_1.10_T5_release/middleware/utils/syschannel/syschannel_host/wal/wal_net.c
+++
b/WS53_1.10_T5_release/middleware/utils/syschannel/syschannel_host/wal/wal_net.c
@@ -249,6 +249,8 @@ STATIC osal_void syschannel_init_netdev(osal_u8 type,
oal_net_device_stru *netde
    #endif
        netdev->netdev_ops          = wal_get_net_dev_ops();

+    netdev->needs_free_netdev = false;
+
    oal_netdevice_destructor(netdev)      = wal_net_free_netdev;
    oal_netdevice_ifalias(netdev)          = OSAL_NULL;
    oal_netdevice_watchdog_timeo(netdev)   = 5;

```

## 4.5 爱科微AIC8800M方案

爱科微(Aicsemi)的AIC8800M方案同样提供了详尽的开发文档，其主控侧的开发模式与海思方案类似，也采用类以太网驱动和私有消息通信的架构。

- 主机侧Wi-Fi驱动

```
sysdrv\drv_ko\wifi\aic8800_netdrv
```

- 主机侧应用开发代码

```
sysdrv\drv_ko\wifi\aic8800_netdrv\custom_msg
```

- 私有消息通信协议描述

```
SDK\aic8800-sdk\docs\UserGuide\Fhostif-UserGuide.pdf
```

```
SDK\aic8800-sdk\docs\Functions\Protocol\CustomMsg-Protocol.pdf
```

- Wi-Fi MCU功能开发文档

```
SDK\aic8800-sdk\docs\Functions\WiFi\WiFiConsole.pdf
```

```
SDK\aic8800-sdk\docs\UserGuide\GenericUserGuide-v1.2.03.pdf
```

## 4.6 高拓ATBM6441/6461方案

主控侧跟海思方案一致，参考海思方案。

设备侧开发参考：

- SDK源码

FTP（**请联系FAE获取**）/11-Linux平台/WIFI低功耗方案/高拓ATBM/ATBM6441\_SDK.7z

- 文档参考

FTP/11-Linux平台/WIFI低功耗方案/高拓ATBM/ATBM6441/文档资料/软件说明文档

ATBM\_IOT\_SUPPLICANT功能函数API使用说明.pdf

高拓讯达ATBM\_IOT\_SUPPLICANT使用说明.pdf

开发步骤：

首先把FTP上的/11-Linux平台/WIFI低功耗方案/高拓ATBM/atbm6461.rk.tar.bz2放到sdk的sysdrv/drv\_ko/wifi目录下，然后修改如下：复用ATBM6441的配置

```
sysdrv/drv_ko/wifi$ git diff .
diff --git a/Makefile b/Makefile
index 7601424..64a65ff 100644
--- a/Makefile
+++ b/Makefile
@@ -76,6 +76,7 @@ endif

 ifneq ($(findstring $(RK_ENABLE_WIFI_CHIP),"ATBM6441"),)
     @make -C atbm6441/
+    @make -C atbm6461/
 endif
```

编译会得到：

```
sysdrv/drv_ko/wifi/atbm6461/driver_install/atbm6461_wifi_sdio.ko
```

编译应用：

```
arm-rockchip831-linux-uclibcgnueabi-hf-gcc tools/tools.c -o server
```

```
arm-rockchip831-linux-uclibcgnueabi-hf-gcc tools/tools_cli.c -o cli
```

测试：

```
#加载驱动
insmod /oem/usr/ko/cfg80211.ko
insmod /data/atbm6461_wifi_sdio.ko

#把cli和server放到data目录下
cd /data/
./server &

./cli scan
./cli scan_results
./cli set_network ssid wwc2
./cli set_network key 1234abcd
```

```
./cli set_network key_mgmt WPA2
./cli enable_network
./cli list_networks

./cli status
wifi_mode=STA
wpa_state=ACTIVE
ssid=wwc2
ip_address=192.168.50.66
mac_address=04:42:1a:b4:e7:78
ip_mask=255.255.255.0
gate_way=192.168.50.1
OK

ifconfig wlan0 192.168.50.66
echo nameserver 192.168.50.1 > /etc/resolv.conf
route add default gw 192.168.50.1 wlan0

# ping www.baidu.com
PING www.baidu.com (157.148.69.151): 56 data bytes
64 bytes from 157.148.69.151: seq=0 ttl=52 time=39.631 ms
64 bytes from 157.148.69.151: seq=1 ttl=52 time=39.542 ms
```

## 5. 调试指导

- **首要步骤：**在开始调试前，请务必详细阅读Wi-Fi模块原厂提供的SDK开发文档。实践表明，多数问题源于对Wi-Fi模块自身特性和工具链的不熟悉。
- **问题定位：**遇到问题时，首先要界定问题属于主控侧还是Wi-Fi模块侧。例如，若ifconfig看不到wlan0节点，则可能是主控驱动加载问题；若无法连接路由器，则需通过模块日志来分析。如果不确定，可通过Redmine系统提交问题单以获得支持。
- **主控侧调试：**请参考Rockchip标准文档《Rockchip\_Developer\_Guide\_Linux\_Wi-Fi\_BT\_CN.pdf》进行排查。
- **Wi-Fi模块侧调试：**请务必连接并打开Wi-Fi模块的调试串口，这是获取模块内部运行状态、日志信息和排查问题的最关键手段。
- **固件烧录：**确保已搭建好Wi-Fi模块的编译环境，并已成功编译及烧录固件到模块中。一个常见的错误是模块内没有有效固件，导致主控无法正常识别和通信。